National University of Computer and Emerging Sciences

Fast School of Computing Spring 2026

CS-4031 Compiler Construction

Quiz # 1 (Solution)

Section: _____

Time = 10 minutes Name: _____

26/01/2026 Roll No: _____

Mark the following statements as either True or False. [CLO-1]

1A

1. True – Object code or intermediate files may be produced.
2. True – Unlinked object code cannot usually execute.
3. False – Interpretation causes runtime overhead.
4. False – Logic errors may still exist.
5. True – Machine code runs directly on hardware.
6. True – Assembly closely maps to machine instructions.
7. False – Platform, compiler, and OS also matter.
8. True – Compilation is done before execution.
9. True – Interpreters execute line by line.
10. False – Some hardware understanding is still useful.
11. True – Comments or optimized code may generate none.
12. False – Hardware architecture must also match.
13. False – Linkers resolve external references too.
14. False – Compiler is not needed at runtime.
15. True – Machine dependence reduces portability.

1B

1. True – Logic errors need reasoning, not rules.
2. False – High-level languages are easier to modify.
3. False – Interpreters combine translation and execution.
4. False – Many interpreters generate intermediate code.
5. True – Compilers don’t verify correctness of logic.
6. True – Abstraction speeds development.
7. False – Closer to hardware = more control.
8. False – Platform dependency still exists.
9. False – Interpreters favor flexibility, not speed.
10. True – Compilation time improves execution speed.
11. True – High-level languages are more concise.
12. True – Entire program must compile first.
13. False – Logic errors are mostly runtime issues.
14. True – Different architectures produce different binaries.
15. True – Ease of use vs control trade-off.

1E

1. True – Logical correctness is not guaranteed by compilation.
2. True – Tokens must be identified before parsing.
3. True – Different architectures produce different executables.
4. True – Fewer passes mean more language restrictions.
5. True – Assembly must be assembled before execution.
6. True – IR helps target multiple machines.

7. False – Compiler is not needed at runtime.
8. True – Semantic errors go beyond grammar.
9. True – Code generation depends on target machine.
10. False – Forward references are harder, not impossible.
11. True – Interpretation mixes translation and execution.
12. False – CPU architecture must also match.
13. False – Optimization preserves program semantics.
14. True – Extra passes need extra storage.
15. True – Symbol tables store identifier information.

1F

1. False – Machine code runs directly on hardware.
2. True – Front end is mostly machine independent.
3. True – Compilation is done before execution.
4. True – Earlier analyses detect errors first.
5. False – Linkers also resolve external references.
6. True – First pass gathers information for second pass.
7. True – Ease of programming vs hardware control trade-off.
8. True – Assembly closely maps to machine instructions.
9. True – Efficiency and portability often conflict.
10. True – Fewer passes mean more language restrictions.
11. False – One-pass compilers can generate object code.
12. False – Phases can be combined or overlapped.
13. True – Errors appear close to the failing statement.
14. False – Syntax correctness ≠ semantic correctness.
15. False – CPU architecture must also match